

Knowledge Graphs Synthesis from Text: A Language Model Approach

Kaushal Patil **Divya Rajput** **Arushi Saran** **Ayush Parikh** **Soham Patel**
kspatil@usc.edu drajput@usc.edu arushisa@usc.edu ayushspa@usc.edu sohampat@usc.edu

1 Abstract

Our project proposes an automated pipeline to build accurate and concise domain-specific knowledge graphs from unstructured text. We employ overlapping text chunking for context preservation, SciBERT embeddings to eliminate redundant information, and dual-mode triplet generation using direct inference and retrieval augmented generation (RAG). We also introduce an automated benchmarking framework that evaluates entity connectivity, relation clarity, and overall graph quality. Large Language Model-based evaluations confirm that our method outperforms existing approaches, delivering scalable, validated knowledge graphs without human intervention.

2 Introduction

In many domains, turning unstructured text into accurate, scalable knowledge graphs remains challenging: manual annotation is slow and automated tools often produce redundant, noisy triples. To address this, we built an end to end pipeline that slices documents into overlapping chunks to preserve context, applies SciBERT based vector filtering to eliminate near duplicates, and generates relations through both direct inference and RAG. Recognizing the lack of a standard evaluation, we embedded an LLM driven benchmarking framework that scores entity connectivity, relation clarity and graph coherence without human oversight. Together, these elements produce concise, high quality graphs that adapt to new domains and update in real time, streamlining the path from raw text to actionable knowledge.

3 Related Work

Our project builds on a range of recent efforts to automate knowledge graph (KG) construction using large language models (LLMs). [Huguet Cabot and Navigli \(2021\)](#) introduced REBEL, a BART-based

seq2seq system that performs end-to-end entity detection, relation labeling, and triple formation in a single pass laying foundational work for prompt-driven relationship extraction from text. [MontageBai \(2024\)](#) extended this concept to a specialized domain, using Qwen2 LLM prompts to parse over 100,000 papers on framework materials (MOFs, COFs, HOFs) and generate a Neo4j KG with millions of nodes and edges, integrated into a RAG QA system.

[Kommineni et al. \(2024\)](#) proposed a six-component pipeline spanning data collection, competency question (CQ) generation, ontology creation (leveraging PROV-O), CQ answering, KG construction, and evaluation with human-in-the-loop checks to ensure quality. [Chen and Bertozzi \(2023\)](#) demonstrated a lightweight alternative: extracting keywords as nodes and using graph Laplacian learning to weight co-occurrence edges, combined with hybrid search for richer retrieval. Finally, [Lairgi et al. \(2024\)](#) presented iText2KG, an incremental, zero-shot framework that avoids post-processing by distilling documents into entities and relations in real time and visualizing the evolving KG in Neo4j.

A detailed tabular comparative study of the various papers has been added in Fig.7.

4 Problem Description

Despite the existing methodologies, constructing domain-specific KGs from unstructured text remains a persistent challenge. Manual annotation, while accurate in theory, is highly labor-intensive and prone to inconsistencies that can cascade through the KG, undermining its reliability. Automated approaches often suffer from overgeneration, introducing contextually irrelevant triples that dilute the KG’s utility. These pipelines frequently trade semantic richness for processing speed, fail-

ing to enforce domain-specific constraints or validate relationships with sufficient depth. A major barrier to progress is the absence of standardized evaluation metrics: performance indicators vary widely by domain, schema, and implementation, thus without consistent ground truths or scalable, objective evaluation frameworks, current methods rely heavily on expensive human validation. As a result, practitioners are left without the robust extraction techniques or rigorous, comparable validation criteria needed to effectively deploy, assess, and maintain high-quality knowledge graph pipelines. None of the papers studied comprehensively addresses these critical weaknesses.

Our architecture will preserve fine-grained context through overlapping chunking, eliminate noise via vector-based deduplication, enforce semantic accuracy with multi-pass LLM refinement, and embed an automated LLM-driven benchmarking framework yielding a unified, fully automated pipeline that will be both scalable and rigorously validated.

5 Methodology

The following section reviews the foundational components and methodological innovations underlying the pipeline architecture illustrated in Fig. 1. It surveys the design choices and frameworks that informed each stage of our automated KG generation process. Additionally, we also mention the design principles that were considered behind forming the benchmarks.

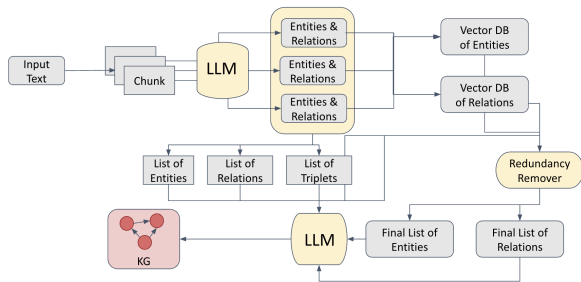


Figure 1: Pipeline for Automated Generation of KG

5.1 Preprocessing through chunking

We begin by splitting each paper’s text into overlapping snippets of 1,000 characters (with a 100-character overlap) so that no key idea is cut off. Most existing KG pipelines don’t address LLM context-window limits: they either extract at sentence level so REBEL can miss relations spanning

longer passages or feed full abstracts without systematic chunking, risking loss of nuance when documents exceed token caps; AutoKG uses small fixed windows with no overlap, and iText2KG splits text sequentially, dropping any relation that crosses a boundary. In contrast, we break each paper into overlapping 1,000-character snippets (with 100 characters of overlap), ensuring every sentence and subtle clue appears within the LLM’s manageable context window. This targeted chunking lets the model focus on smaller, coherent text blocks, preventing critical facts from being buried in lengthy passages and yields far richer, more complete extraction of relationships than approaches that feed entire documents at once.

5.2 Structure Entity and Relation Extraction

Each text chunk extracted from the document is processed by a language model using a carefully crafted prompt that incorporates few-shot examples and encourages chain-of-thought reasoning. The prompt also specifies a structured output format, guiding the model to return a JSON-formatted list comprising entities, relations, and corresponding triplets. The outputs generated from individual chunks are then aggregated to form an initial, comprehensive list of entities, relations, and triplets. However, this merging process may introduce redundancy, as semantically equivalent entities and relations, whether explicitly or implicitly expressed, can appear multiple times across different chunks. Therefore, a redundancy elimination step is essential to ensure that the resulting knowledge graph remains concise and non-redundant.

5.3 Vector Database Creation and Redundancy Removal

To facilitate redundancy detection, we first embed every candidate entity and relation using the SciBERT model¹. Specifically, each term in the initial lists is converted into a fixed-length vector by feeding it through SciBERT’s encoder. These vectors are then inserted into two separate Annoy indices one index for entity embeddings and another for relation embeddings using Spotify’s Annoy library² for approximate nearest-neighbor search. By leveraging Annoy, we avoid the computationally expensive nested loop that would otherwise be required to compare each term against every other term in the list.

¹<https://github.com/allenai/scibert>

²<https://github.com/spotify/annoy>

Once both indices are populated, we iterate over each item in the original entity and relation lists and perform a nearest-neighbor query to retrieve its ten most similar vectors. For each neighbor returned, we compute the cosine similarity score with the query vector. If this score exceeds 0.90, the neighbor is deemed semantically equivalent and is excluded from the final consolidated list; neighbors with similarity at or below 0.90 are retained as distinct. This threshold-based filtering effectively removes both explicit duplicates and near-duplicates cases where different surface forms convey the same underlying concept ensuring that our knowledge graph remains non-redundant.

An important advantage of this architecture is its support for incremental updates: newly identified entities or relations can be embedded and appended to the existing Annoy indices without necessitating a full re-indexing. This design aligns with the “freshness” criterion (Trajanoska et al. (2023)) for dynamic knowledge graphs, allowing real-time integration of new data while preserving the efficiency and precision of redundancy elimination.

5.4 Knowledge Graph Creation

Once the final lists of entities and relations have been assembled, they are used along with supporting context to drive knowledge-graph construction via a language model guided by a few-shot prompt designed around our quality-evaluation criteria such as accuracy, coherence and completeness. We evaluated two distinct reference-base strategies for triplet generation:

- **Triplet-Based Reference:** The initial set of extracted triplets serves as the sole knowledge base. The LLM is prompted to infer additional relations exclusively from this compact repository.
- **RAG-Based Reference:** The entire research paper functions as the retrieval corpus within a RAG framework. At query time, the LLM retrieves relevant passages from the full text to inform the generation of new triplets.

5.5 Benchmark - Quality Evaluation Principles

We devised a comprehensive set of quality-evaluation principles that cover Entities, Relations and entire Knowledge Graphs. At the entity level we require dense connections between nodes, full capture of contextual information and preservation

of every key attribute. At the relation level we insist that each triple be concise, that synonyms are mapped and ambiguities removed and that no two triples contradict one another. At the graph level we enforce non redundancy by eliminating duplicate or semantically equivalent triples and we measure concentrated ness so the graph remains tightly focused on its core concepts. Applying these criteria allows us to objectively assess, compare and improve any knowledge graph.

For evaluation, we relied on LLM-as-a-judge to avoid the need for human feedback and evaluation, and to get a standardized rating. We split the above-mentioned principles into different prompts, utilized few-shots training and prompted Gemini-2.0-Flash to give a numerical rating out of 10 for each of the principles. These ratings were then aggregated to get a final rating. The Fig. 6 in the Results section captures all outputs.

6 Results and Discussion

This section details the outcomes of the automated knowledge graph constructed through our pipeline and evaluates multiple knowledge graphs using the benchmarking principles we developed.

6.1 Automated Knowledge Graph Output

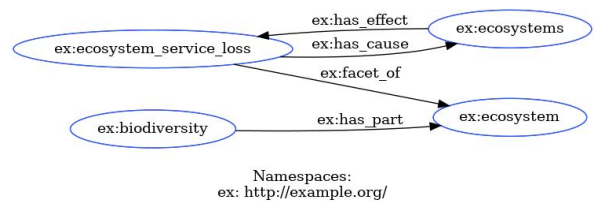


Figure 2: Output for REBEL

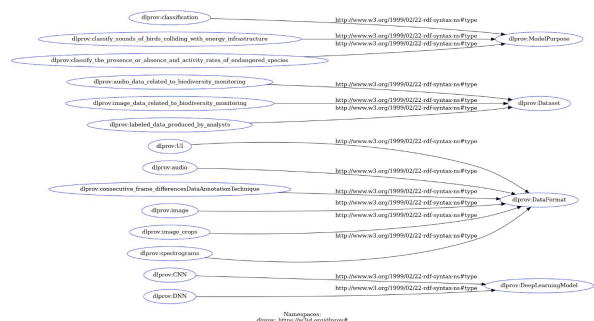


Figure 3: Output for Humans to Machines Paper

Figures 2 (Huguet Cabot and Navigli, 2021) and 3 (Kommineni et al., 2024) illustrate the limitations of prior approaches, where the resulting knowledge graphs fail to encode salient entity-relation pairs

and suffer from structural sparsity and inconsistency. These deficiencies undermine their utility for reasoning tasks, as the extracted representations lack both relational completeness and topological coherence.

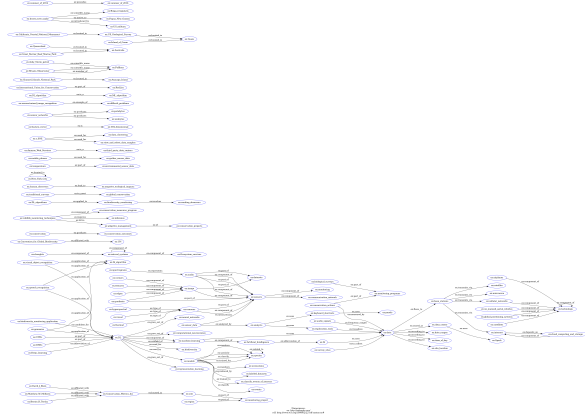


Figure 4: Gemini-RAG based Knowledge graph



Figure 5: Llama-RAG based Knowledge graph

Fig. 4 and Fig. 5 present a significant improvement over the results from previous papers we reviewed. These results were able to identify nuanced entities and relations within the text that the previous papers failed to extract. Comparative experiments demonstrate that the RAG-based approach for Gemini (41 triplets) and Llama (13 triplets) yield a substantially larger set of valid triplets and therefore richer relational connectivity than the triplet-only method. This finding highlights the advantage of leveraging broader document context, in line with the “completeness” principles of high-quality knowledge graphs.

6.2 Benchmark evaluation of generated Knowledge Graphs

In the table below, using our qualitative benchmarking principles, we have evaluated KG outputs for 2 of the research papers we had studied, added outputs of our pipeline for both Gemini and Llama based model, and also specified the results for our RAG based approach for comparison. These results were generated using a research paper with over 4000 words as an input. Overall, REBEL scored the lowest, with specially low score for disambiguation and density. One point to note is that it

got full score for Relation: Non-Contradiction, but this was because the number of triplets extracted is too small and simple for any contradictions. Our architecture overall outperformed the research papers we had reviewed. This was possible because chunking the input helped capture the entire context, and the redundancy removal resulted in more concise, non-repetitive and more meaningful triplets being generated. Using Gemini with the RAG based approach performed the best overall, having an average rating of **7.63**. This is because by passing the input context again to the LLM, more relevant triplets could be extracted.

Benchmark	Entity			Relation			Knowledge Graph		Final Rating
	Density	Contextual Information	Attributes	Concise	Disambiguation	Non-contradiction	Non-redundancy	Conciseness	
REBEL	3	5	4	7	6	9	7	6	5.00
From Humans to Machines Creating Knowledge Graph	3	5	4	7	6	9	7	6	5.88
Our Approach (Llama-3)	5	7	6	7	5	9	8	7	6.00
Our Approach (Gemini)	6	7	6	7	7	9	9	7	7.25
Our Approach + RAG (Llama-3)	6	7	5	6	7	9	9	8	7.13
Our Approach + RAG (Gemini)	7	7	6	7	7	9	9	9	7.63

Figure 6: Gemini-Flash based KG evaluation for each model

7 Conclusion

Our results demonstrate a novel end-to-end, LLM powered pipeline that can transform raw unstructured text into rigorously validated knowledge graphs. By preserving cross-sentence context through overlapping chunking, enforcing semantic consistency via embedding-based deduplication, and enriching graph structure with both triplet-only and RAG-augmented extraction, we achieve a level of accuracy and completeness that surpasses existing automated approaches. Moreover, our LLM as judge benchmarking framework offers a reproducible, domain-aware evaluation methodology enabling fair comparisons across models. Altogether, our work lays a strong foundation for scalable, domain-agnostic KG construction, offering a versatile solution for transforming unstructured data into structured, actionable knowledge representations.

8 Future Work

In future, we plan to augment our RAG framework with multi-hop reasoning: once an initial set of triplets has been generated, we will index the partially constructed graph itself and allow the language model to retrieve from both the original document corpus and the evolving graph. This dual retrieval strategy is designed to uncover deeper, multi-step relationships, thereby enhancing the structural consistency, interpretability, and analytical depth of the resulting knowledge graph.

9 Individual Contribution

- Kaushal: Worked and researched on [Kommineni et al. \(2024\)](#), obtained relevant results for the same, explored evaluation principles for KG. Designed the new pipeline for the study and implemented the vector DB creation and redundancy elimination module.
- Divya: Conducted a research on [Lairgi et al. \(2024\)](#), analyzing model architecture, performance and implemented pre-requisites for KG, executed the final phase of our proposed approach creating a KG, utilizing both Llama and Gemini language models.
- Arushi: Reviewed the [Huguet Cabot and Navigli \(2021\)](#) paper for REBEL architecture, used REBEL to obtain KG results, explored KG benchmarks, built prompts for KG benchmarking using LLM and evaluated KGs generated for multiple approaches to draw a comparison
- Soham: Researched [Chen and Bertozzi \(2023\)](#) to highlight AutoKG’s core approach, then performed knowledge benchmarking across multiple LLM and LLM + RAG configurations to evaluate their performance and graph quality.
- Ayush: Studied the [MontageBai \(2024\)](#), analyzing its methodology and implementing the first stage in the pipeline involving chunking and data preprocessing.

Yassir Lairgi, Ludovic Moncla, Rémy Cazabet, Khalid Benabdeslem, and Pierre Cleau. 2024. [itext2kg: Incremental knowledge graphs construction using large language models](#). *Preprint*, arXiv:2409.03284.

MontageBai. 2024. Kgfm: A knowledge graph for framework materials. Available at <https://github.com/MontageBai/KGFM> (accessed on Month Day, Year).

Milena Trajanoska, Riste Stojanov, and Dimitar Trajanov. 2023. [Enhancing knowledge graph construction using large language models](#). *Preprint*, arXiv:2305.04676.

References

- Bohan Chen and Andrea L. Bertozzi. 2023. [Autokg: Efficient automated knowledge graph generation for language models](#). In *2023 IEEE International Conference on Big Data (BigData)*, pages 3117–3126.
- Pere-Lluís Huguet Cabot and Roberto Navigli. 2021. [REBEL: Relation extraction by end-to-end language generation](#). In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 2370–2381, Punta Cana, Dominican Republic. Association for Computational Linguistics.
- Vamsi Krishna Kommineni, Birgitta König-Ries, and Sheeba Samuel. 2024. From human experts to machines: An llm supported approach to ontology and knowledge graph construction. *arXiv preprint arXiv:2403.08345*.

Appendix

Comparative Parameter	iText2KG	Framework Materials KG	REBEL	AutoKG	From Human Experts to Machines
Goal	General-purpose KG from text	Domain-specific KG from literature	Relation extraction via generation	Lightweight graph for retrieval	Semi-automated KG construction with expert validation
Domain Scope	Domain-independent	Framework materials (MOFs, COFs, HOFs)	General-purpose (fine-tunable)	Domain-adaptive with tuning	Domain-flexible; grounded via competency questions
Use of LLMs	Zero-shot LLM (no fine-tuning)	Qwen2 LLM (pre-trained)	Fine-tuned BART model	Prompted LLM for keywords	LLMs for scoring and guidance, RAG for answering CQs
Architecture / Pipeline	Modular: 4 components	Qwen2 + Neo4j + RAG QA	Seq2seq encoder-decoder	Embeddings + Laplacian scoring	6-component pipeline
Extraction Granularity	Incremental extraction	Paper-level parsing	Sentence-level generation	Chunk-level keyword graph	Concept/relationship extraction from competency questions (CQs)
Training Requirements	No training needed	No training	Requires fine-tuning	No training	Partial automation
Output Format	Structured KG	Neo4j KG	Triplet strings	Light graph + hybrid search	Ontology-grounded KG (e.g., PROV-O)
Query Interface	Not specified	Cypher & natural language (RAG)	Not specified	Vector + graph-based search	RAG-enabled interface informed by ontology and CQs
Strengths	Plug-and-play, precise, modular, adaptable	Scalable, automated, QA with citation	End-to-end learning, accurate with tuning	No training, fast, few hallucinations	Structured, integrates ontologies, balances LLM + human input
Weaknesses	Prompt-sensitive, noisy input issues, threshold tuning	Domain-locked, no accuracy metrics, no competency checks	Black-box generation, sentence-level limitations	Prompt-sensitive, mostly qualitative evaluations	LLM variability, integration complexity, requires human validation

Figure 7: Comparative Study of Literature Review